# MainLayoutComponent - Implementación Completada ☑

## Descripción

El `MainLayoutComponent` es el contenedor principal de la aplicación BiblioKeep. Proporciona:

- **Sidebar Colapsable**: Responsivo, se convierte en drawer en dispositivos móviles
- **Header Responsivo**: Con el nombre de la app "BiblioKeep"
- **Navegación**: Enlaces a Dashboard, Biblioteca y Préstamos
- **Router Outlet**: Área de contenido donde se cargan las vistas
- **100% Tailwind CSS**: Sin dependencias de librerías de componentes
- **Accesibilidad**: ARIA labels y navegación por teclado

## Estructura de Archivos

```
src/app/shared/components/main-layout/
├── main-layout.component.ts      # Componente standalone con lógica
├── main-layout.component.html    # Template con Tailwind
├── main-layout.component.css     # Estilos de animación
└── GUIDE.md                      # Documentación del componente
```

## Características Implementadas

### 1. Sidebar Responsivo

- **Desktop (≥1024px)**: Sidebar fijo a la izquierda
- **Mobile (<1024px)**: Drawer deslizable con overlay semi-transparente
- **Cierre automático**: Al hacer clic en un enlace de navegación

### 2. Navegación

```
navItems = [
  { label: 'Dashboard', route: '/dashboard', icon: 'BarChart3' },
  { label: 'Mi Biblioteca', route: '/library', icon: 'BookOpen' },
  { label: 'Préstamos', route: '/loans', icon: 'Hand' },
];
```

### 3. Estado con Signals

```
sidebarOpen = signal(false);  // Controla visibilidad del sidebar
```

```
  toggleSidebar(): void { ... }  // Alternar estado
  closeSidebar(): void { ... }   // Cerrar explícitamente
```

### 4. Diseño Tailwind

- **Sidebar**: `bg-slate-900 text-white` con bordes sutiles
- **Header**: `bg-white shadow-sm` con contraste
- **Contenido**: `bg-gray-50` para mejor legibilidad
- **Transiciones**: `transition-transform duration-300` para fluidez

## Rutas Configuradas

```
// app.routes.ts
{
  path: '',
  component: MainLayoutComponent,
  children: [
    { path: 'dashboard', component: DashboardComponent },
    { path: 'library', component: LibraryComponent },
    { path: 'loans', component: LoansComponent },
    { path: '', redirectTo: 'dashboard', pathMatch: 'full' }
  ]
}
```

## Componentes Temporales Creados

Mientras se desarrollan los componentes de features, se incluyen placeholders:

- **DashboardComponent** - En construcción
- **LibraryComponent** - En construcción
- **LoansComponent** - En construcción

## Mejoras Futuras (Lucide Angular)

Una vez instalado `lucide-angular`, reemplazar los SVG genéricos con iconos específicos:

```
npm install lucide-angular
```

Entonces actualizar el template:

```
<i [lucide]="item.icon"></i>
```

## Testing en Navegador

La aplicación está ejecutándose en `http://localhost:4200` con:

- [x] Compilación sin errores
- [x] Hot reload habilitado
- [x] Estructura completa lista para features
- [x] Responsive design (prueba redimensionando la ventana)

## Próximos Pasos

1. **Autenticación**: Crear `AuthService` en `core/services`
2. **HTTP Interceptor**: Inyectar JWT tokens automáticamente
3. **Componentes Atómicos**: Button, BookCard, StatsWidget
4. **Features Completas**: Lógica de Dashboard, Library y Loans

---

**Estado**: [x] Marco visual completado | Backend: Listo | Frontend: En progreso